TEST AUTOMATION ENGINEERING

E-Commerce End-to-End Test Automation Manual

Technical Execution Specification Guide Using Selenium WebDriver and Java

Document Version	v1.1.0 (Updated)
Target Architecture	Java / Selenium WebDriver 4.x
Execution Target	Google Chrome / ChromeDriver
Classification	Technical Reference Specification

1. Test Automation Strategy & Operational Matrix

Validating multi-stage user actions requires structured element synchronization, visibility validation rules, and specialized UI interaction patterns to maintain framework resilience and stability.

Workflow Phase	Technical Approach & Strategy	Selenium WebDriver API Mechanism
Authentication	Scale browser viewport; use explicit sync checks prior to key execution.	`ExpectedConditions.visibilityOfElementLocated`
Query Processing	Isolate and interact with functional query nodes to route key phrases.	`sendKeys()` and `click()` operations
Metadata Verification	Fetch rendering validations for structural layout details (Title, Price, Ratings).	`isDisplayed()` boolean assertions
UX Interaction	Simulate native hardware mouse-hover actions over media container arrays.	`Actions.moveToElement().perform()` builder
Transaction Validation	Execute continuous click operations on verified transaction nodes.	Sequential `ExpectedConditions.elementToBeClickable`

> **Synchronization Requirement:** Do not implement hardcoded script delays such as `Thread.sleep()`. Use dedicated `WebDriverWait` monitoring strategies to effectively absorb unexpected server latency or asynchronous asset loading processes.

2. Sequential Implementation Steps

The core automation workflow is organized below into logical, sequential implementation steps:

Step 1 Secure System Portal Authentication

Maximize the browser screen to ensure complete visibility of structural UI layers, access the secure entry point URL, wait for element initialization, and pass valid profile authentication parameters.

```
driver.manage().window().maximize();
driver.get("https://example-ecommerce.com/login");
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username"))).sendKeys("v
alid_user");
driver.findElement(By.id("password")).sendKeys("valid_password");
driver.findElement(By.id("loginBtn")).click();
```

Step 2 Targeted Product Discovery

Ensure the primary text field component is visible, type the target evaluation term (e.g., `"laptop"`), and submit the lookup parameters down the pipeline.

```
WebElement searchBox =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("search")));
searchBox.sendKeys("laptop");
driver.findElement(By.id("searchBtn")).click();
```

Step 3 Product Metadata Integrity Verification

Isolate the target matching anchor node link from the results view, trigger a click event, and retrieve immediate presentation verifications for key metadata fields.

```
WebElement productLink = wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//
a[contains(text(),'Laptop')]")));
productLink.click();

WebElement title =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("productTitle")));
WebElement price = driver.findElement(By.id("productPrice"));
WebElement rating = driver.findElement(By.id("productRating"));

System.out.println("Title Displayed: " + title.isDisplayed());
System.out.println("Price Displayed: " + price.isDisplayed());
System.out.println("Rating Displayed: " + rating.isDisplayed());
```

Step 4 Simulating Hover for Advanced Zoom Features

Use the specialized user interaction API class to move the virtual pointer coordinate matrix directly over the main image element, triggering the application's hover-activated zoom asset layout.

```
WebElement productImage = driver.findElement(By.id("productImage"));
actions.moveToElement(productImage).perform();

WebElement zoomedImage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("zoomed-image")));
System.out.println("Image Zoomed: " + zoomedImage.isDisplayed());
```

Step 5 Checkout Processing and Final Validation Steps

Confirm the structural rendering profiles of actionable transaction elements. Proceed to execute consecutive ordering steps through to the checkout view screen.

```
WebElement addToCartBtn = driver.findElement(By.id("addToCart"));
System.out.println("Add to Cart Button Visible: " + addToCartBtn.isDisplayed());

WebElement buyBtn = driver.findElement(By.id("buyNow"));
System.out.println("Buy Button Visible: " + buyBtn.isDisplayed());
buyBtn.click();

WebElement checkoutBtn =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("checkout")));
System.out.println("Checkout Button Visible: " + checkoutBtn.isDisplayed());
checkoutBtn.click();
```

3. Integrated Execution Script Specification

The integrated production-ready implementation file below bundles all workflow operations into a single test runner class, using explicit synchronization parameters and standard clean driver termination controls.

```
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class ECommerceFlowSuite {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
```

```
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        Actions actions = new Actions(driver);

        try {
            driver.manage().window().maximize();
            driver.get("https://example-ecommerce.com/login");


wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username"))).sendKeys("v
alid_user");
            driver.findElement(By.id("password")).sendKeys("valid_password");
            driver.findElement(By.id("loginBtn")).click();

            WebElement searchBox =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("search")));
            searchBox.sendKeys("laptop");
            driver.findElement(By.id("searchBtn")).click();

            WebElement productLink =
wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//
a[contains(text(),'Laptop')]")));
            productLink.click();

            WebElement title =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("productTitle")));
            WebElement price = driver.findElement(By.id("productPrice"));
            WebElement rating = driver.findElement(By.id("productRating"));

            System.out.println("Title Displayed: " + title.isDisplayed());
            System.out.println("Price Displayed: " + price.isDisplayed());
            System.out.println("Rating Displayed: " + rating.isDisplayed());

            WebElement productImage = driver.findElement(By.id("productImage"));
            actions.moveToElement(productImage).perform();

            WebElement zoomedImage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("zoomed-image")));
            System.out.println("Image Zoomed: " + zoomedImage.isDisplayed());

            WebElement addToCartBtn = driver.findElement(By.id("addToCart"));
            System.out.println("Add to Cart Button Visible: " +
addToCartBtn.isDisplayed());

            WebElement buyBtn = driver.findElement(By.id("buyNow"));
            System.out.println("Buy Button Visible: " + buyBtn.isDisplayed());
            buyBtn.click();

            WebElement checkoutBtn =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("checkout")));
            System.out.println("Checkout Button Visible: " + checkoutBtn.isDisplayed());
```

```
            checkoutBtn.click();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            driver.quit();
        }
    }
}
```